Common Layouts

Example 1: Column Layout

```
<View style={styles.container}>
   <View style={style.box1}></View>
   <View style={style.box2}></View>
   <View style={style.box3}></View>
</View>

const styles = StyleSheet.create({
    container: {
        flex: 1,
        justifyContent: 'space-between',
        alignItems: 'center',
        backgroundColor: '#eee',
    },
   box1: {
       width: 100,
       height: 100,
       background: 'red'
   },
    box2: {
       width: 100,
       height: 100,
       backgroundColor: 'green'},
    box3: {
        width: 100,
        height: 100, backgroundColor: 'blue' }
```

Example 2: Row Layout

```
container: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
}
```

Example 3: Responsive Layout with flex

```
<View style={{flexDirection: 'row'}}>
  <View style={{ flex: 1, backgroundColor: 'red',
                 height: 100}}/>
  <View style={{flex: 2, backgroundColor: 'blu
                 height: 100}}/>
</View>
```

Example 4: Wrapping Boxes

```
<View style={{flexDirection: 'row', flexWrap: 'wra
            }}>
{[...Array(8)].map((_, i) => {
  <View key={i} style={{ width: 80,
                         height: 80,
                         margin: 5 }}/>
  )})}
</View>
```

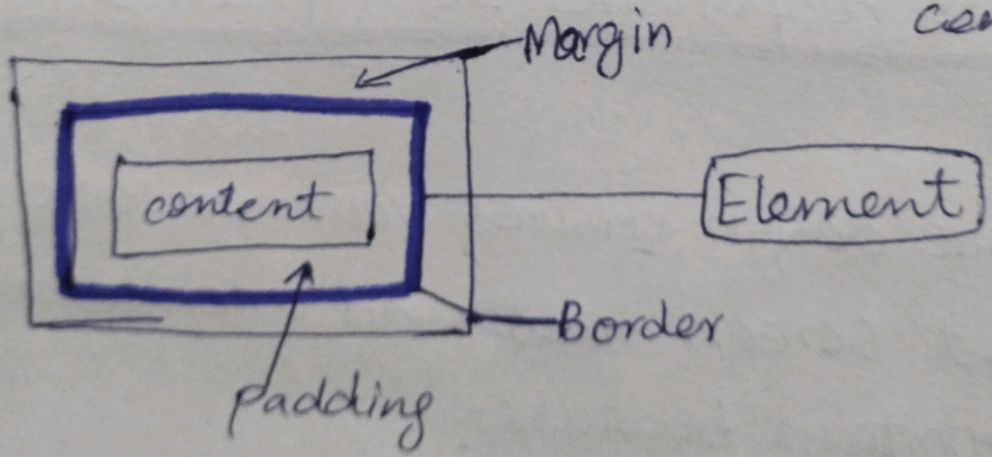
Nesting Views

```
<View style={style.outer}>
  <View style={styles.innerTop}>
    <Text>Top section</Text>
  </View>
  <View style={styles.innerBottom}>
    <Text>Bottom Section</Text>
  </View>
</View>
```

Difference b/w margin & padding

Margin: Space oustide element (b/w elements)

Padding: Space inside element (b/w border and content)



What property makes a child element fill parent space?

→ The `flex` property.

```
const styles=StyleSheet.create({
  parent:{
    flex: 1, // Parent takes all space
  },
  child:{
    flex:1, // child fills parent
  },
});
```

Why is responsive design Imp in mobile apps?

→ Responsive design ensures your app looks good on different screen sizes and orientations.

→ Its important because:

- Device Variety (Screen and resolution)
- User Experience
- Professional Appearance
- Orientation Changes

→ How to achieve responsive layout

- Use flexbox for flexible layout
- Use percentage widths
- Use `flex: 1` for filling space
- Test on multiple devices
- Use responsive units